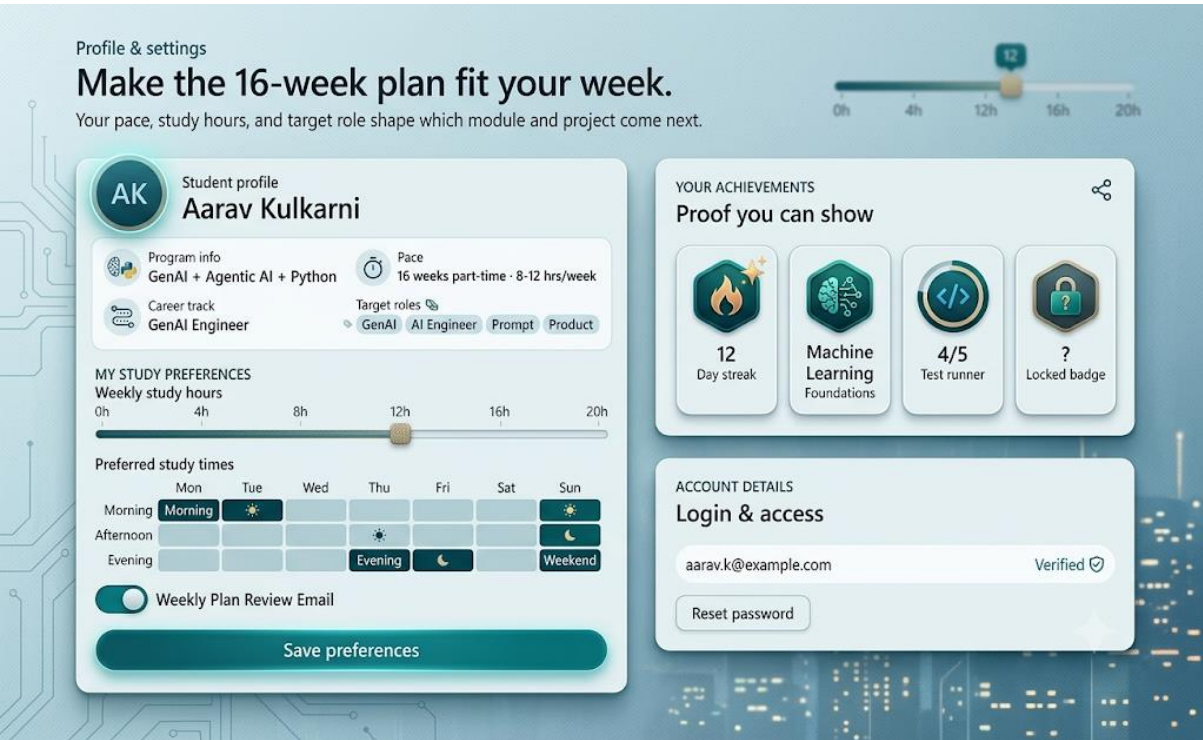
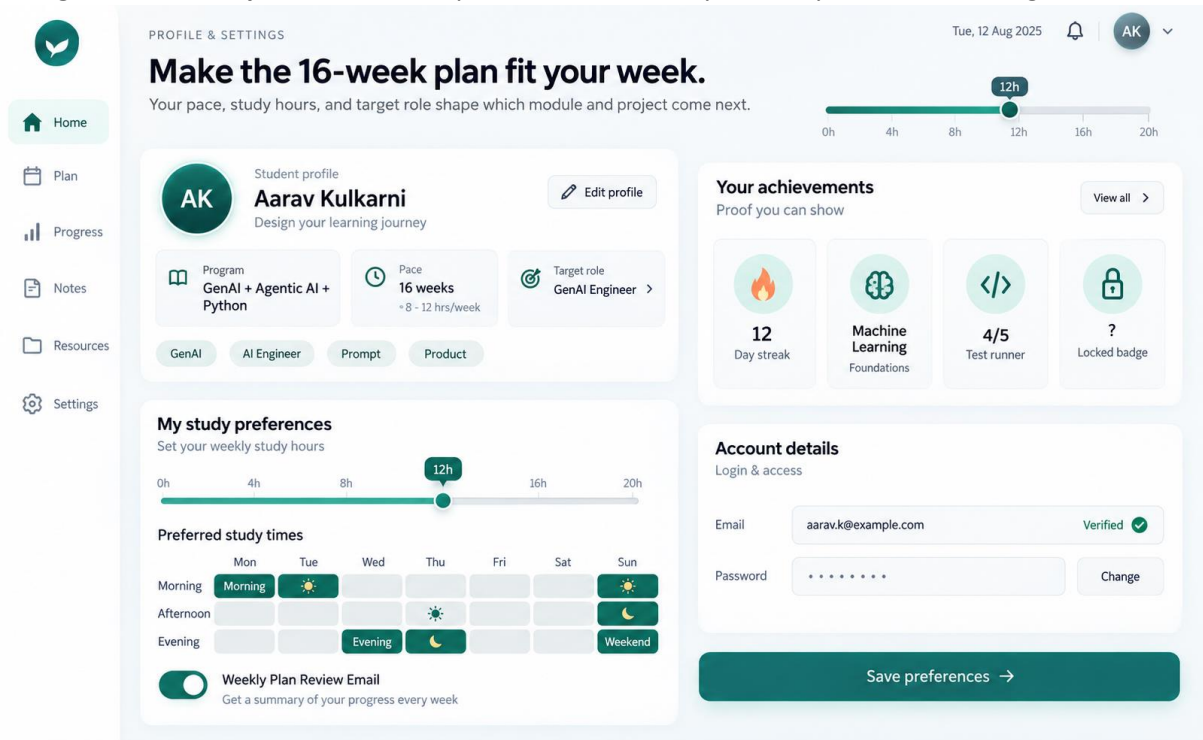


Rename the **Portfolio** page to **Profile** or **MySpace** to make it feel more personal and student-focused.
Reorganize the **card placements** to improve visual hierarchy, usability, and overall navigation.



Home page navigation has an error: clicking a specific subject/module always redirects to **Module 1** instead of the selected module.

Students should be able to **preview locked modules** with only headings, subheadings, and basic information visible. The actual content should remain **locked until the required previous modules are completed**, preventing students from bypassing the learning path.

The banner shows the current milestone as 'Module 01 - Python Foundations for AI' with a 'Continue Lesson' button and a progress indicator '0/20'. Below this is 'The GenAI Mastery Path: From Python to Agents', a sequence of five phases: Phase 1 Foundations (Modules 01-04), Phase 2 Knowledge Apps (Modules 05-08), Phase 3 Agents (Modules 09-13), Phase 4 Domain & Production (Modules 14-17), and Phase 5 Portfolio & Career (Modules 18-20). Each phase has a corresponding icon and a brief description of the deliverable. Below the phases are three project cards: Phase 1 Project: LLM Chatbot, Phase 2 Project: PDF RAG Q&A, and Phase 3 Project: Tool-using Agent.

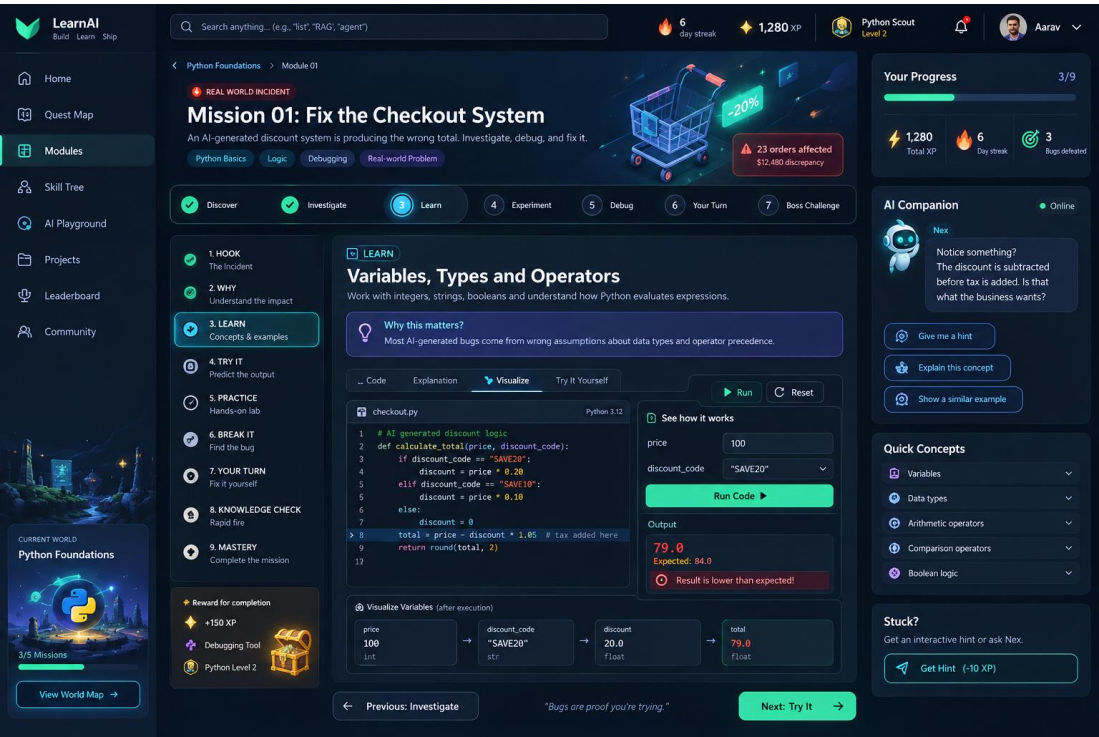
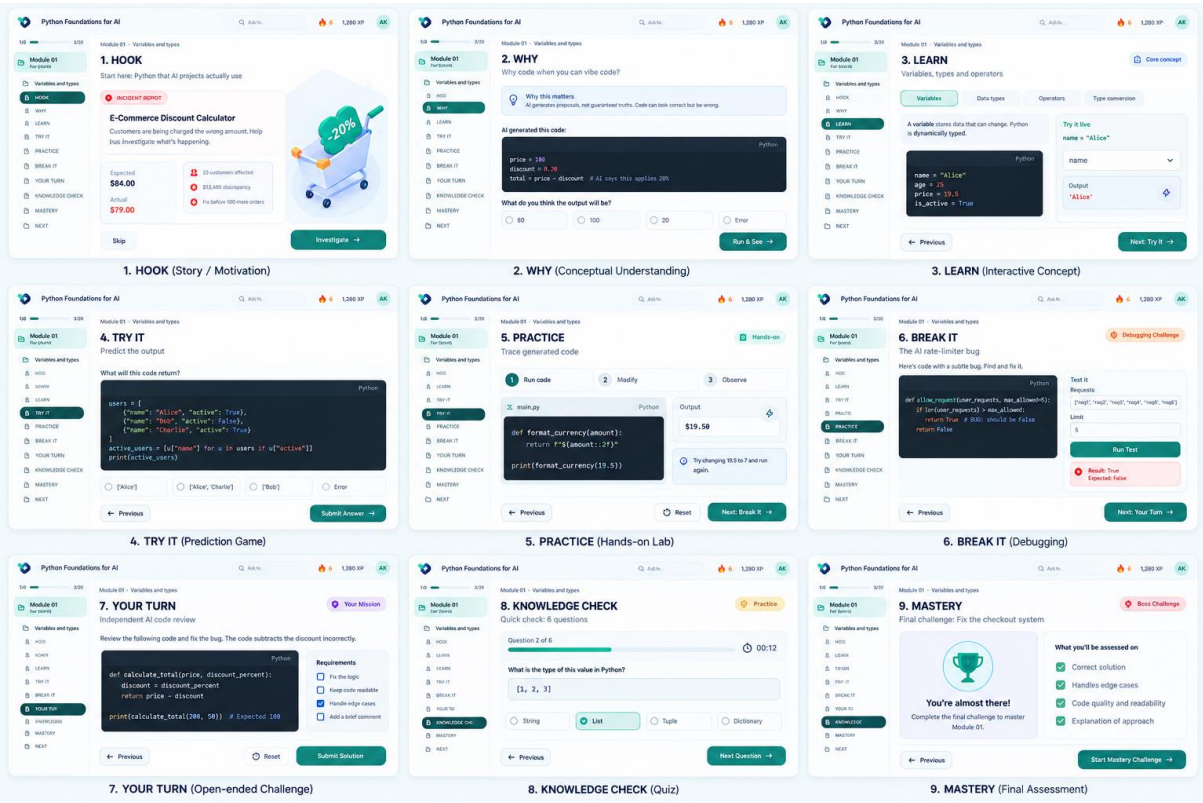
The interface shows the course title 'Module 01: Python Foundations for AI' and a description: 'Build the programming discipline required for AI development: clean syntax, structured programs, reliable data handling, and a professional Python workflow.' It includes a 'Continue lesson' button and a 'See all 20 modules' button. A progress bar indicates '0 of 20 modules done' and 'Next topic: Variables and types'. Below this are statistics: 20 total modules, 5 phases, 16 weeks recommended pace, and a 'Career ready' badge. The 'The learning path' section shows five phases with their respective modules and topics. Phase 1 (Foundations) is active, showing topics like Python basics, data structures, problem solving, and clean code habits. Phases 2 through 5 are locked. The 'Featured modules' section highlights 'LLM Chatbot' (Modules 01-05), 'PDF RAG Q&A' (Modules 07-09), and 'Tool-using agent' (Modules 10-11). On the right, there are motivational quotes, a 'Keep going!' reminder, a 'Your progress' section with a checklist, and a 'Next up' section for 'Lesson 1: Variables and types'.

The tabs are **too text-heavy and feel static**. Add more **infographics, visualizations, interactive examples, and animations**, while making navigation smoother and more intuitive.

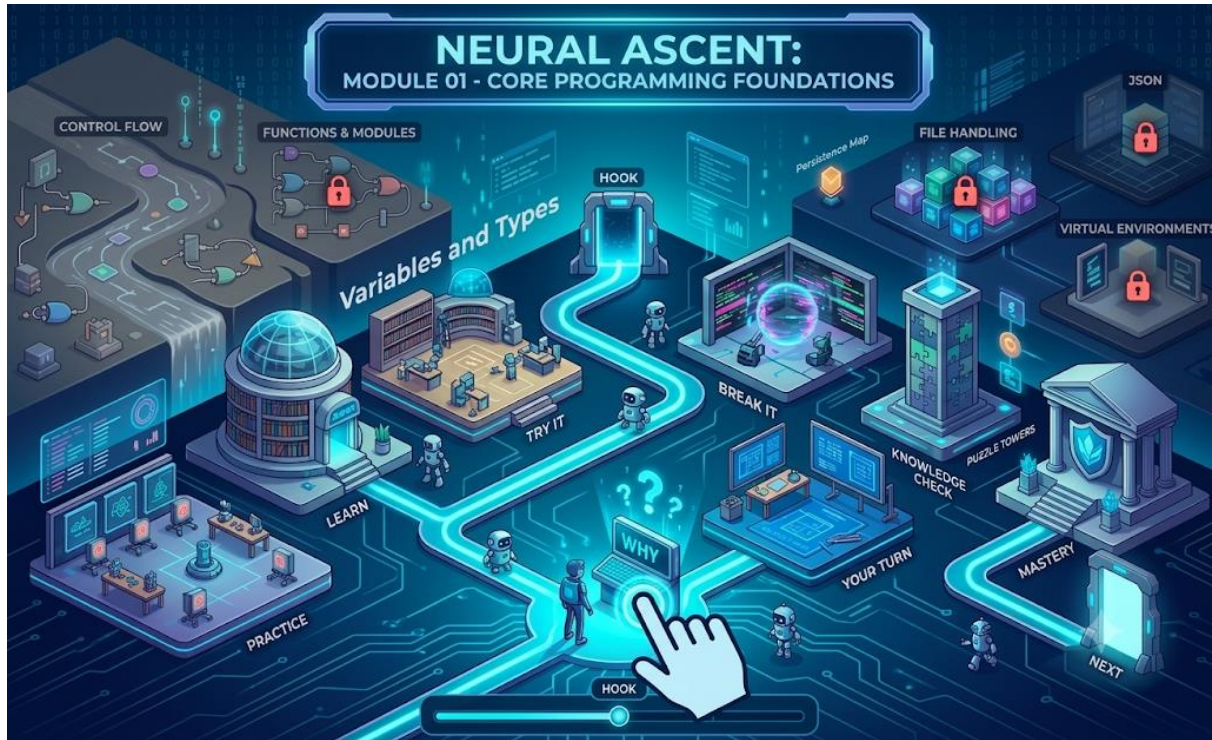
Replace the current **old-school, folder-based architecture** with a modern, engaging learning flow.

The goal is for students to feel:

“I’m exploring and interacting with the concept, not reading a textbook on a screen.”



One of the gamification concepts inspired by Ralph Breaks the Internet to keep it interactive



The **Practice** tab currently feels too static and boring. It should be redesigned to feel **interactive, engaging, and hands-on**, where students actively experiment with code rather than simply reading and answering questions.

Add more **interactive challenges, real-time code execution, instant feedback, visual responses, progressive difficulty, hints, and small rewards** to maintain engagement.

The goal is for students to feel:

“I’m actually practicing and experimenting, not just completing another exercise.”

The screenshot shows the 'AI Code Detective' practice module in the LearnAI interface. The main content area displays a code editor with a Python function `count_messages` and a table of sample test cases. The test results section shows 3 out of 5 tests passing. The right sidebar includes a 'Run your code' button and a 'Keep going!' message.

Module 01
Python Foundations

AI Code Detective
Concept: AI-generated code review • Est. time: ~15 mins • Difficulty: Beginner

Your task
Review the AI-generated code below and fix the logical issues so that it correctly counts only non-empty messages.

Requirements

- ✓ Ignore empty strings ("")
- ✓ Ignore None values
- ✓ Return the correct count
- ✓ Keep the function signature

Run your code
Run Tests

Test results
3 / 5 passing

- ✓ Test 1: Basic messages
Input: ["hello", "world"]
Expected: 2 | Got: 2
- ✓ Test 2: With empty strings
Input: ["hi", "", "there"]
Expected: 2 | Got: 2
- ✓ Test 3: With None values
Input: ["hello", None, "ok"]
Expected: 2 | Got: 2
- ✗ Test 4: All empty
Input: ["", "", None]
Expected: 0 | Got: 1
- ✗ Test 5: Mixed edge cases
Input: [None, "a", "", "b", None, ""]
Expected: 2 | Got: 3

Sample test cases

CASE	INPUT	EXPECTED OUTPUT	DESCRIPTION
01	["hello", "world"]	2	Two valid messages
02	["hi", "", "there"]	2	Ignore empty string
03	["hello", None, "ok"]	2	Ignore None values
04	["", "", None]	0	No valid messages
05	[None, "a", "", "b", None, ""]	2	Mixed edge cases

Module Progress
5 / 10 50%

Stuck?
Get a hint from Nex

Keep going!
3 out of 5 tests are passing. Fix the failing cases to complete this practice.
Need help? Show me a hint

Complete Practice

The screenshot shows the 'AI Code Detective' practice module in the LearnAI interface, redesigned to be more interactive and engaging. The main content area displays a code editor with a Python function `count_messages` and a table of test cases. The test results section shows 3 out of 5 tests passing. The right sidebar includes a 'Your Progress' section with a progress bar and a 'Skill Gain' section with XP rewards.

Module 01
Python Foundations

AI Code Detective
Review the AI-generated code below and fix the logical issues so that it correctly counts only non-empty messages.

Code Editor
solution.py

```
def count_messages(messages):  
    """  
    Return the number of valid messages in a list.  
    A valid message is a non-empty string.  
    """  
    return sum(1 for message in messages if message)
```

Test Cases
3 / 5 passing

#	Input	Expected	Your Output	Status
01	["hello", "world"]	2	2	✓ Pass
02	["hi", "", "there"]	2	2	✓ Pass
03	["hello", None, "ok"]	2	2	✓ Pass
04	["", "", None]	0	2	✗ Fail
05	[None, "a", "", "b", None, ""]	2	3	✗ Fail

Your Progress
3 / 10 30%

Skill Gain

- Python Basics +5 XP
- Debugging +5 XP
- Code Review +0 XP

Keep going!
You're 30% through Module 01.

Run Tests **Submit Solution**

The screenshot shows the LearnAI dashboard for a user named Aarav. The main section is titled "Six builds hiring managers can inspect" and lists six project milestones. The first two are completed, while the others are not started. A "Next step" section indicates the user has the Python and API foundations and needs to complete a proof of work. A "Portfolio progress" section shows 33% completion (2/6 projects). A "Project checklist" lists requirements like clear README, pinned dependencies, and no API keys. An "AI Mentor" section offers guidance. A "Submit Project Repository" section allows users to submit their GitHub repository for evaluation.

Next step
You have the Python and API foundations. Next proof of work is a cited PDF Q&A (Gate B) and a bounded agent with traces (Gate C).

Project milestones
Choose at least 3 projects. Each project includes a checklist and will be reviewed automatically.

Milestone	Project Name	Description	Checklist	Status	Submitted On
01	LLM Chatbot (Python + API)	Streaming chat with system prompt, history, temperature control, and token logging. Maps to modules 01-05.	☒ README ☒ Requirements ☒ Demo script	Completed	Submitted on 12 Sep 2025
02	Semantic Search System	Index notes with embeddings and query by natural language with scores and metadata. Maps to module 06.	☐ Index script ☐ Query CLI ☐ Similarity metrics	Completed	Submitted on 14 Sep 2025
03	RAG-based PDF Q&A	Ingest PDFs, chunk, retrieve, and answer with citations plus an abstain path. Maps to modules 07-09.	☐ Sample PDFs ☐ Evaluation set ☐ Faithfulness notes	Not started	
04	AI Agent with Tools	Function calling agent with at least two tools, a max step count, and logged traces. Maps to modules 10-11.	☐ Tool implementations ☐ Trace logs ☐ Safety note	Not started	
05	Automation Assistant	Bounded workflow with human confirmation before side effects. Maps to modules 11-12 and 17.	☐ Workflow diagram ☐ Demo video / GIF ☐ Risk analysis	Not started	
06	Multi-Agent System	Supervisor plus two specialists with round limits and a failure mode analysis. Maps to modules 12-13 and 17.	☐ Architecture diagram ☐ Demo transcript ☐ One-page analysis	Not started	

Verification & Proof
Submit Project Repository
Paste the public GitHub repository link for the selected project. We'll automatically verify the structure and checklist.

<https://github.com/username/project-name> [Submit for Evaluation](#)

- Repository should contain the **source code, README, setup instructions, requirements/dependencies, and demo/evidence**.
- On submission, the system should automatically check the repository for **required files, dependencies, basic functionality, and test cases**.
- Projects should have **automated test cases** wherever possible.
- Hidden test cases should be used to verify actual functionality rather than only checking submitted files.
- AI should evaluate the project against predefined criteria such as **functionality, code quality, architecture, error handling, documentation, and requirements**.
- The learner should receive **pass/fail status, test results, AI feedback, and areas to improve**.
- If the project fails evaluation, the learner should be able to **fix and resubmit**.
- Successful completion should mark the project as **Verified/Completed** and unlock the next project or learning stage.
- Store the **submission history, evaluation results, feedback, attempts, and completion status** for the learner's portfolio.
- Completed projects should become part of the learner's **verified portfolio/proof of work**.

Simple project flow

Learn → Build → Test → Submit GitHub → Automated Evaluation → AI Review → Feedback → Fix/Resubmit → Verified → Portfolio

For the UI

Each project should clearly show:

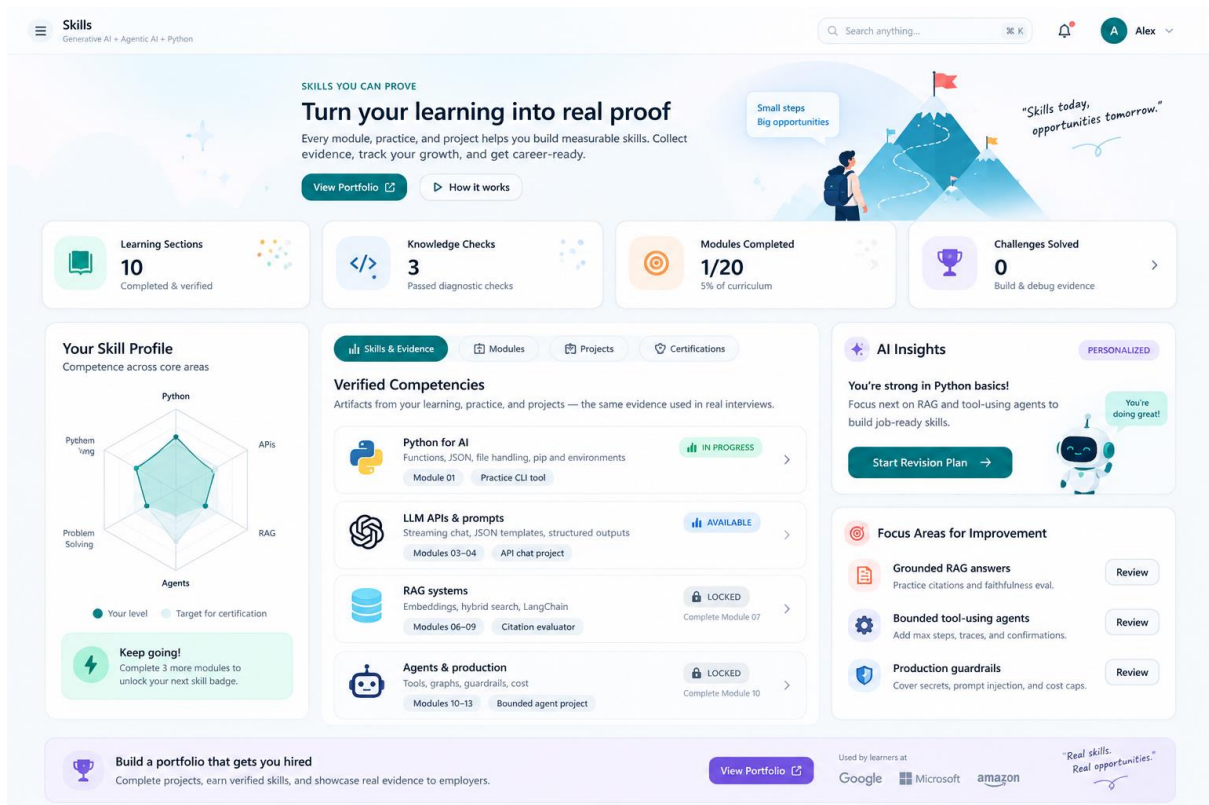
Build requirements → Progress → Submit Repository → Evaluation Status → Test Results → AI Feedback → Resubmit/Complete

The **Skills** tab should show the student's actual learning journey and career track in detail, so they can clearly understand **where they are, what they have achieved, what they are currently learning, and what comes next.**

The experience should use **psychological motivation principles** to keep students engaged without feeling overwhelming. Show visible progress, milestones, skill growth, achievements, upcoming unlocks, and clear next steps so students continuously feel a sense of **progress, curiosity, achievement, and momentum.**

The goal is to make students feel:

"I can see exactly how far I've come, what I'm becoming good at, and what I need to do next."



Overall comments:-

- The platform should feel **modern, interactive, and gamified**, while maintaining a clean and professional UI.
- Reduce **text-heavy screens** and communicate concepts through **infographics, visualizations, icons, animations, and interactive elements**.
- Make the learning journey feel like a **progressive track/mission**, rather than a traditional course or LMS.
- Improve **navigation consistency** so every button, module, and subject takes users to the correct destination.
- Add **clear progress, milestones, skill levels, achievements, XP/streaks, and unlocks** to keep students motivated.
- Keep gamification **subtle and meaningful**—focused on learning achievements rather than excessive badges or animations.
- Practice should be more **hands-on and interactive**, with live coding, challenges, instant feedback, hints, and real-time results.
- Learning modules should have **less scrolling and reading**, with concepts broken into smaller interactive experiences.
- Locked modules should provide a **preview of headings/subheadings**, while preventing students from bypassing prerequisites.
- Projects should follow a clear **Build → Submit → Test → AI Evaluation → Feedback → Resubmit → Verify** flow.
- The Profile/MySpace area should showcase the student's **skills, progress, projects, achievements, and career readiness** in a visually engaging way.
- Maintain a **consistent design system** across all screens—same typography, spacing, buttons, cards, icons, colors, and interaction patterns.
- Add **smooth transitions and micro-interactions** to make the platform feel responsive without becoming distracting.
- Overall, avoid the feeling of an **old-school course/document platform** and create an experience where students feel they are **actively progressing, building, experimenting, and improving**.